

UNIVERSITATEA BABEȘ-BOLYAI CLUJ-NAPOCA
FACULTATEA DE MATEMATICĂ ȘI INFORMATICĂ
SPECIALIZAREA INFORMATICĂ ÎN LIMBA
ROMÂNĂ

LUCRARE DE LICENȚĂ

Adresarea Problemei Imaginilor Îmbătrânite și Degradate

Conducător științific
Dr. Miholca Diana-Lucia

Autor
Țurcă Vasile-Ionuț

2024

**BABEȘ-BOLYAI UNIVERSITY CLUJ-NAPOCA
FACULTY OF MATHEMATICS AND COMPUTER
SCIENCE
SPECIALIZATION COMPUTER SCIENCE IN
ROMANIAN**

DIPLOMA THESIS

Addresing the Problem of Aged and Degraded Images

**Supervisor
Dr. Miholca Diana-Lucia**

*Author
Țurcă Vasile-Ionuț*

2024

ABSTRACT

The super resolution algorithms have been discussed right from the 80s. With the passage of time, it has become a problem that has been given more and more attention, both due to the benefits that such algorithms can bring to the resources used to store or transmit images, as well as the evolution of cameras, increasing the desire to have the old images brought back to life or the usage into the medical domain.

In this thesis we discuss the increasing resolution of the old images that suffer from aging or are made using less performing devices, from the perspective of the deep learning approach.

The prior work on that problem has evolved at the same time with the artificial intelligence models. There were a lot of techniques used in the past: Pre-Upsampling Super Resolution, which uses traditional methods to refine the resulting image; Post-Upsampling Super Resolution, where the feature extraction plays a crucial role; Residual Networks; multiple different architectures based on the existing models that have proven to be definitely game changers.

In this thesis we will focus on the most modern methods discovered until today. The usage of the Generative Models for this task results into more optimal perceptual quality at the level of the produced photos, managing to replicate even realistic textures, not just better defined areas. We started from the existing models and used Ghost Bottlenecks in order to optimize the speed of the inferring procedure, obtaining a neural network with similar results compared to the existing ones, and studied the possible contribution of the use of Attention Mechanisms. On a system with RTX3050 GPU, the model manages to improve 5000 images in 5.27s versus 8.79s obtained by EnhanceNet and 10.98s obtained by Super-resolution Generative Adversarial Network, the model from whose architecture we started.

The model without Attention Mechanisms was integrated into an iOS native mobile application to prove the usability on the less powerful devices. The application offers the base features needed to use the model at its full potential.

Contents

1	Introduction	1
1.1	Purpose and Targets	1
1.2	Original contribution	2
1.3	Thesis content	2
2	Theoretical aspects	4
2.1	Learning algorithms	4
2.1.1	Supervised Learning	4
2.1.2	Semi-supervised Learning	4
2.1.3	Unsupervised Learning	5
2.1.4	Reinforcement Learning	5
2.2	Artificial Neural Networks	5
2.2.1	Structure	5
2.2.2	Artificial Neuron	6
2.2.3	Activation Functions	6
2.2.4	Learning in Artificial Neural Networks	7
2.3	Convolutional Neural Networks	10
2.3.1	Architecture overview	10
2.3.2	Depthwise Separable Convolution	11
2.3.3	Ghost Bottlenecks	11
2.3.4	Attention Mechanisms	13
3	Related work	15
3.1	Classical Deep Learning-based Methods	15
3.1.1	Super-Resolution Convolutional Neural Network	15
3.1.2	Enhanced Deep Residual Network	17
3.2	GAN-based methods	18
3.2.1	Super-resolution Generative Adversarial Network	18
3.2.2	EnhanceNet	20
3.2.3	Enhanced Super-Resolution Generative Adversarial Networks	22
3.3	Conclusion	23

4	Ghost Enhancer Mobile Application	25
4.1	Super Resolution Enhancer	25
4.1.1	Model Architecture	25
4.1.2	Dataset	26
4.1.3	Training and deployment	27
4.1.4	Experimental Results	27
4.1.5	SWOT Analysis	30
4.2	Mobile Application	31
4.2.1	Technologies	31
4.2.2	Application Architecture	33
4.2.3	Application Flows	36
4.2.4	Quality Assurance	39
5	Conclusions and further possible improvements	43
	Bibliography	45

Chapter 1

Introduction

With the passage of time, cameras have become more and more efficient, producing clearer and clearer images. The pictures are also viewed on screens that render more and more complex images with better and better clarity. Increasing the resolution of the images would lead to the creation of images that respect the latest quality standards from some old and outdated ones. Also, the generation of improved images must be done in a way as simple as possible for the people who want to do it, for example using an application that would facilitate the improvement of both physical and digital images.

This thesis focuses on introducing a novel approach to enhance old images by increasing their resolution by using a General Adversarial Network (GAN) deep learning method. By using Ghost Bottlenecks, it is obtaining the results comparable with those of the state-of-the-art methods, with a significantly less time needed for processing. This thesis also contains a study of the Attention Mechanisms on our new network. Furthermore, it presents a mobile application that can be used to obtain the inferred images.

1.1 Purpose and Targets

This thesis aims to discover and improve a Super-Resolution model for obtaining an image enhancer that is able to increase the resolution of an old image to the resolution of the images made these days. In order to achieve that milestone, there is a list of objectives we set to follow, one by one, until the finish line:

1. **Search for the state-of-the-art methods that had a significant result on this problem:** We made a comprehensive review of the existing models, in chronological order, highlighting the strengths and innovations they brought to the table. We also analyzed their weaknesses and did it in a glance in the widest possible perspective.

2. **Optimizations methods for the neural networks:** We investigated the benefits of several architectures and techniques used on other problems to obtain a more powerfull network.
3. **Design and implement of the Ghost Networks:** We started from the existing models, made several architectural changes and applied different optimizations techniques to obtain GhostSRGAN and SA GhostSRGAN, our networks.
4. **Model performance review:** We trained our models on a large dataset, containing multiple images and evaluated their performance compared with the state-of-the-art methods
5. **Develop an iOS application for the people who wants to resurrect the photos:** We created a minimalistic and easy to use mobile application to prove the portability of the model and allow the people to use it.

The purpose of reaching these targets is to bring a contribution to the research made for the Super Resolution task and to provide an easy to use application.

1.2 Original contribution

We started from the architecture of an existing state-of-the-art GAN-based deep learning model and we have used some innovative layers, Ghost Bottlenecks and Multi-Head Attention Layers, managing to obtain a faster model, that is able to infer images at the quality of the model we started from. We integrated that model in an iOS mobile application, made with focus on the user experience, being very easy to use even by the non-technical people.

1.3 Thesis content

We started presenting the theoretical foundations that are the foundation stones for building the discussed solution. In Chapter 2 we made a comprehensive review of the learning algorithms, Artificial Neural Networks and we presented how they do the magic behind the problems they solve. In the last part of that part we analyzed the Convolutional Neural Networks and some layers that proved themselves to be very effective in solving complex tasks very efficiently.

Chapter 3 is a comprehensive review of the existing solutions, with pros and cons. We present the important changes of the SRCNN, ESDR, SRGAN, EnhanceNet and ESRGAN that influenced their success.

Chapter 4 contains the description of our models, the architecture and the functionalities of the mobile application. First, we were discuss the parts we brought starting from the SRGAN model and the differences between GhostSRGAN and SA GhostSRGAN. After that, we offer a brief review of the used dataset. We explain how we trained the models and their results compared with the GAN-based methods. We are also offering a SWOT Analysis of the models. Second half of that chapter is reserved for the mobile applications. It starts presenting the technologies we used and continues presenting the architecture. The third part is about the interactions between the user and the app. We added relevant explanations about how we tested the application.

The thesis is ending with the conclusion and further possible improvements starting from the models and the discussed application.

During the preparation of this work the author used chatGPT in order to reword some phrases to make them easier to understand. After using this tool/service, the author reviewed and edited the content as needed and takes full responsibility for the content of the thesis.

Chapter 2

Theoretical aspects

This chapter explores the foundational concepts of Artificial Neural Networks, with a focus on Convolutional Neural Networks. We will examine different learning methodologies. Additionally, we will discuss the training processes for neural networks, delve into the Attention Mechanism in Deep Learning, and analyze several model architectures that are pertinent to enhancing image resolution.

2.1 Learning algorithms

Seen as a whole, there are four types of machine learning algorithms: supervised algorithms, semi-supervised algorithms, unsupervised algorithms and reinforcement algorithms, classified according to the way they learn [Mah20].

2.1.1 Supervised Learning

In supervised learning, a model is developed through a process where it is trained using a dataset containing input-output pairs, which are already known [Mah20]. This methodology enables the model to recognize underlying data patterns and, over time, to forecast outcomes more accurately. As the training progresses through various cycles, there is a noticeable improvement in the model's precision and functionality .

2.1.2 Semi-supervised Learning

Semi-supervised learning employs a mix of both labeled and unlabeled data [Mah20]. This approach enhances the learning process, where the presence of labeled data serves as a foundation for the model to interpret and then apply this understanding to the unlabeled portion of the dataset. This method enriches the model's capability to handle data with fewer initial labels.

2.1.3 Unsupervised Learning

Unsupervised learning techniques process datasets without pre-assigned labels, aiming to discover the natural grouping and patterns within [Mah20]. These algorithms autonomously identify similarities and differences in the data, often grouping them into clusters based on their characteristics, without any external input regarding the output.

2.1.4 Reinforcement Learning

Reinforcement learning stands apart from other machine learning frameworks due to its reliance on a trial-and-error approach guided by rewards and penalties [ADBB17]. Models learn and refine their decision-making abilities by engaging with an environment where they receive feedback based on the actions they take. This continuous loop of actions and rewards helps the model to optimize its decision-making processes over time.

2.2 Artificial Neural Networks

Artificial Neural Networks (ANNs) mimic the structure of biological nervous systems through mathematical modeling [Abr05]. They have become increasingly popular because of their ability to perform calculations in parallel and their effectiveness in tasks like image recognition and natural language processing. This chapter offers an overview of ANNs, including their architecture and functionality.

2.2.1 Structure

Artificial Neural Networks (ANNs) consist of several layers, each made up of various interconnected neurons [Abr05]. These layers are categorized into input layers, multiple hidden layers, and an output layer. The input layer serves as the entry point for data into the model. It does not engage in learning through adjustments to weights or other parameters; rather, its function is to prepare or format the incoming data suitably for processing in the deeper layers. The hidden layers, equipped with multiple adjustable parameters, perform complex operations on the data. Finally, the processed data is passed to the output layer, which is tasked with generating the end output and calculating the associated loss.

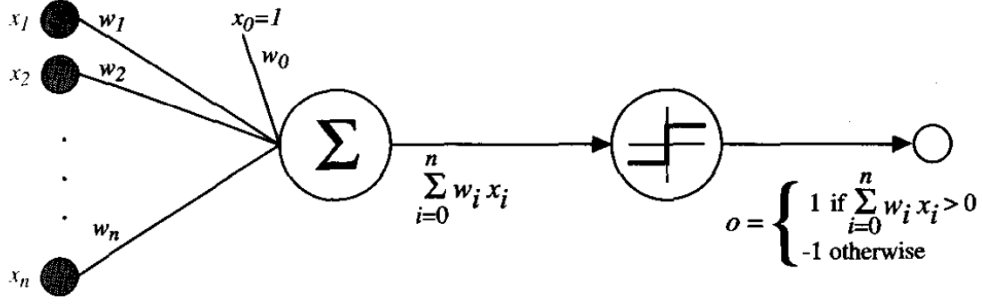


Figure 2.1: A perceptron [MBD⁺90]

2.2.2 Artificial Neuron

The fundamental component of a neural network is known as a perceptron, depicted in Figure 2.1 [MBD⁺90]. The perceptron receives a vector of real-valued inputs, computes a linear combination of these inputs using learned weights, and then produces an output of 1 if the computed value is positive and -1 if it is zero or negative, as described in Equation (2.1).

$$o(x_1, \dots, x_n) = \begin{cases} 1 & \text{if } w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n > 0 \\ -1 & \text{otherwise} \end{cases} \quad (2.1)$$

2.2.3 Activation Functions

Activation functions are vital in Artificial Neural Networks (ANNs) because they determine the output of neurons and introduce non-linearity into the network, allowing it to model complex patterns in data [ADIP21]. Common activation functions include Identity, which outputs the input directly; Step (Heaviside), which outputs a binary value; Bipolar, similar to the Step function but outputs -1 or 1; Sigmoid, which produces a smooth S-shaped curve; Bipolar Sigmoid, an adaptation of the Sigmoid that outputs values between -1 and 1; Hyperbolic Tangent (tanh), which scales the input to between -1 and 1; Hard Hyperbolic Tangent, an approximation of tanh; Absolute Value, which outputs the magnitude of the input; and Cosine, which applies the cosine function to the input. These functions are crucial for enabling the ANN to capture intricate patterns and representations, thereby enhancing its ability to learn from data. Moreover, the choice of activation function can significantly impact the training dynamics and performance of the network, making it a critical aspect of ANN design.

Name	Expression	Range
Identity	$\text{id}(a) = a$	$(-\infty, +\infty)$
Step (Heaviside)	$Th_{\geq 0}(a) = \begin{cases} 0 & \text{if } a < 0 \\ 1 & \text{otherwise} \end{cases}$	$\{0, 1\}$
Bipolar	$B(a) = \begin{cases} -1 & \text{if } a < 0 \\ +1 & \text{otherwise} \end{cases}$	$\{-1, 1\}$
Sigmoid	$\sigma(a) = \frac{1}{1+e^{-a}}$	$(0, 1)$
Bipolar sigmoid	$\sigma_B(a) = \frac{1-e^{-a}}{1+e^{-a}}$	$(-1, 1)$
Hyperbolic tangent	$\tanh(a)$	$(-1, 1)$
Hard hyperbolic tangent	$\tanh_H(a) = \max(-1, \min(1, a))$	$[-1, 1]$
Absolute value	$\text{abs}(a) = a $	$[0, +\infty)$
Cosine	$\cos(a)$	$[-1, 1]$

Table 2.1: A perceptron [ADIP21]

2.2.4 Learning in Artificial Neural Networks

The main strength of the ANNs is their ability to learn based on examples. The learning process is made adjusting the weights and the biases of the neurons [MBD⁺90]. Generally, this process is made using optimization algorithms, such as gradient descent or its derivatives, and a loss function.

Loss functions

A loss function used in a neural network usually compares the output of the network with the ground truth [WMZT20]. There are different loss functions used for various tasks. Commonly used loss functions for classification include 0-1 loss and hinge loss. Also, we will remind the square loss and absolute loss for regression, Square error K-means and distance error for unsupervised learning, mean squared error and mean absolute error for deep learning [TSLL22]. The loss function that quantifies the error in order to compute the adjustments needed to the network parameters [MBD⁺90].

Backpropagation

Backpropagation is one of the most widely used learning algorithm. It belongs to the set of supervised algorithms, consisting in two main steps: the forward step and

the backward step [Abr05].

During an epoch, the input data are passed from the input layer through the hidden layers and through the output layer in order to produce the result of the network [Abr05]. Each layer is built using multiple neurons. The algorithm does not require the adjacent layers to be fully connected. Also, there can be synapses that are skipping one or more layers when they forward the result. Each node receives one or more inputs from the neurons from the previous layers or from the outside world and produces a single output value, sent to a layer that is after the current one or to the outside world.

After the forward step, the backward step is done [Abr05]. Having the final result, the purpose of the backward step is to adjust the weights and the biases of the neurons, using optimization algorithms, such as gradient descent.

Optimizations algorithms

Gradient descent is a fundamental optimization technique widely used in backpropagation to locate a local minimum [HA21]. Its main goal is to optimize the learnable parameters of a neural network to yield the best outcomes.

In the context of backpropagation, identifying a local minimum involves iteratively solving a differential equation by moving in the direction opposite to the gradient of the function at that point [HA21], as shown in the Figure 2.2 . This approach ensures that the parameters adjust in a way that minimizes the error incrementally.

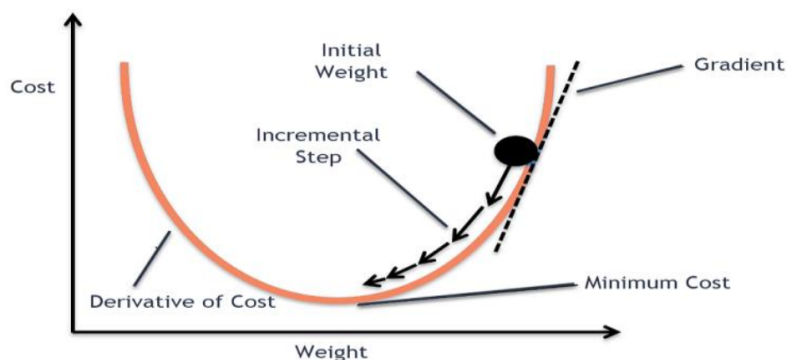


Figure 2.2: Gradient Descent Algorithm [HA21]

Variants of Gradient Descent

There are three main types of gradient descent: Batch Gradient Descent, Stochastic Gradient Descent, Mini-Batch gradient descent, varying in how we iterate over the input data [HA21]. Based on the amount of data processed in a batch, we have a

trade-off between how correct is the parameter update and how much resources do we need to make an update.

Batch Gradient Descent (BGD), also known as vanilla gradient descent, calculates the error for the entire data set and then adjusts the weights [HA21]. Some of the benefits of BGD are efficient computation and balanced error gradient. The main problem is the need to have the entire training dataset in memory in every moment of the training.

Stochastic Gradient Descent (SGD) has proven to be very efficient [HA21]. It randomly selects a few samples from the dataset, calculates the error and changes the network structure. This approach reduces the error by approximating the gradient instead of computing it for the whole dataset.

Mini-batch gradient descent is a variation of SGD and BGD [HA21]. It splits the dataset into smaller batches and update the network learnable parameters for every batch.

Optimization step

The gradient descent changes the weights and biases of the neural network by subtracting a fraction of the computed gradient. This fraction is influenced by a hyperparameter, called learning rate, that controls the step size during the learning, according to the formulas:

$$w_i \leftarrow w_i - \eta \frac{\partial L}{\partial w_i} \quad (2.2)$$

$$b \leftarrow b - \eta \frac{\partial L}{\partial b}, \quad (2.3)$$

where η is the learning rate, and $\frac{\partial L}{\partial w_i}$ and $\frac{\partial L}{\partial b}$ are the gradients of the loss function.

Other optimization algorithms

Starting from the classical types of gradient descent, there are newer optimizations methods, that proved themselves to be very effective in a lot of situations [HA21]. We will remind Momentum, Nesterov Momentum, Adaptive Gradient Descent (AdaGrad), Adaptive Delta (AdaDelta), Root Mean Square Propagation (RMSProp), Adaptive Moment Estimation (Adam), etc.

2.3 Convolutional Neural Networks

Convolutional Neural Networks (CNNs) are very similar to classical ANNs [ON15]. As in the case of ANNs, there is a network with multiple layers, formed from multiple neurons. Still each neuron learns from experience, receiving an input, performing some operations and delivering an output.

The only major difference between CNNs and ANNs is the fact that the first category of neural networks is better at pattern recognition in images, making them more suitable for image-focused tasks [ON15].

A very big limitation of the ANNs is the need of a very large network to process images, resulting in their high computational complexity required for that type of tasks [ON15]. Also, as a result, in the case of CNNs, the risk of overfitting is substantially smaller. The key difference of optimization between CNNs and ANNs is how they work. While in the case of ANNs the information is only processed in the forward direction, the convolutional layers from the CNNs are creating feature maps, focusing more on the information that has to be observed. A notable disadvantage of CNNs is the demand of a larger training dataset.

2.3.1 Architecture overview

Seen as a whole, the generical architecture of a Convolutional Neural Network consists in multiple convolutional layers, pooling layers and fully-connected layers, arranged in an intelligent way, that are processing the input and producing the expected output [ON15].

The general functionality of a CNN can be divided into four parts [ON15]:

1. As in a traditional ANN, the network has an input layer. The input layer introduces the raw data into the network, enabling the hidden layers to process it.
2. The convolutional layers have a significant impact on the network result. It applies a convolution over the input data, which involves sliding a filter with a chosen kernel size over the input, computing the dot product between the input and kernel and producing a feature map. The feature maps are capturing several elements of the input, like shapes, textures, etc. Generally, the feature maps are passed through an activation function to add non-linearity to the learned function.
3. The pooling layers are performing downsampling, reducing the spatial size of the input, reducing the number of parameters within the activation.

4. The fully-connected layers, also known as dense layers, are analyzing the input from the previous layers and compute the output class.

2.3.2 Depthwise Separable Convolution

One of the most recent discoveries in the field of CNNs is represented by the Depthwise Separable Convolutions [Cho17]. This type of convolution works as a classical one, but instead of applying the same filter over all of the channels, it replaces the classical idea with a series of operations that independently look at correlations for each channel.

The depthwise convolution has two major steps [Cho17]. First step is applying the filters on each feature channel on the input. Second step consists into stacking the convolved outputs together using Pointwise Convolution, as shown in Figure 2.3.

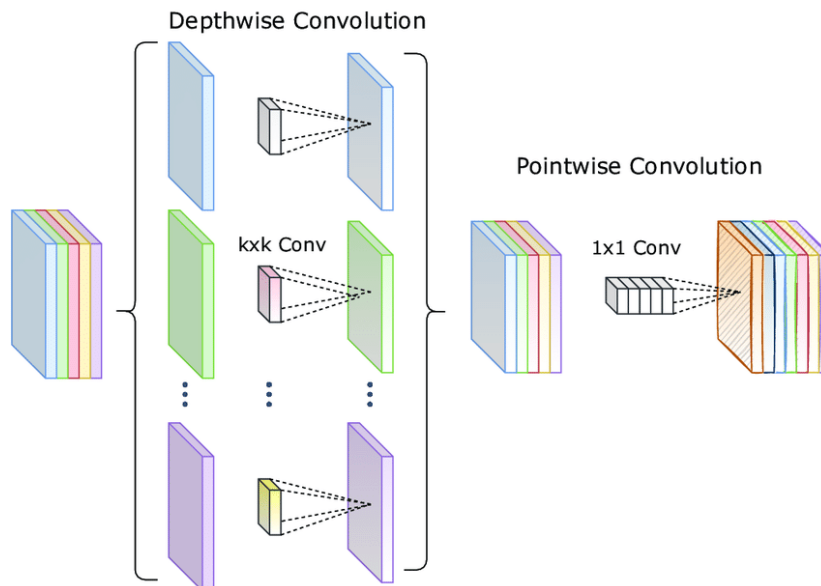


Figure 2.3: Depthwise Separable Convolution [SPY⁺22]

2.3.3 Ghost Bottlenecks

The demand for efficient networks, with a good level of performance, suitable for smartphones, self-driving cars and other mobile devices has led to the creation of the Ghost Module and of the Ghost Bottleneck [WL⁺22]. The authors of GhostNet, the model where that layer were initially used, highlighted that in a neural network, there are some pairs of features that are pretty similar, “like a ghost of each other”. The model switched from the paradigm of avoiding the repetitions to the using them

in a cost-efficient way, highlighting that information that is repeated often contains important data for a comprehensive learning.

Ghost Module

The creation of that type of module started from the observation that in a traditional network there a lot of filters applied to obtain the feature maps [WL⁺22]. The idea behind the GhostModule is to obtain fewer intrinsic features from the input data and to generate the "ghost" feature applying a series of cheaper operations.

The first step made by the GhostModule, as mentioned above, is to generate the intrinsic feature maps [WL⁺22]. May m be the number of the features maps Y' , generated using a primary convolution:

$$Y' = X * f' \quad (2.4)$$

where f' from $\mathbb{R}^{c \times k \times k \times m}$ where represents utilized filters and hyperparameters are the same as an ordinary convolution. To obtain n feature maps (the number of feature maps generated by a Residual Block, for example), the authors apply some cheap operations on every feature map from Y' . The ghost features are obtained using the formula:

$$y_{ij} = \Phi_{i,j}(\psi_j), \quad \forall i = 1, \dots, m, \quad j = 1, \dots, s \quad (2.5)$$

where $n = m * (s + 1)$ and the feature map with the index $is + 1$ is obtained applying the identity function on the feature map i

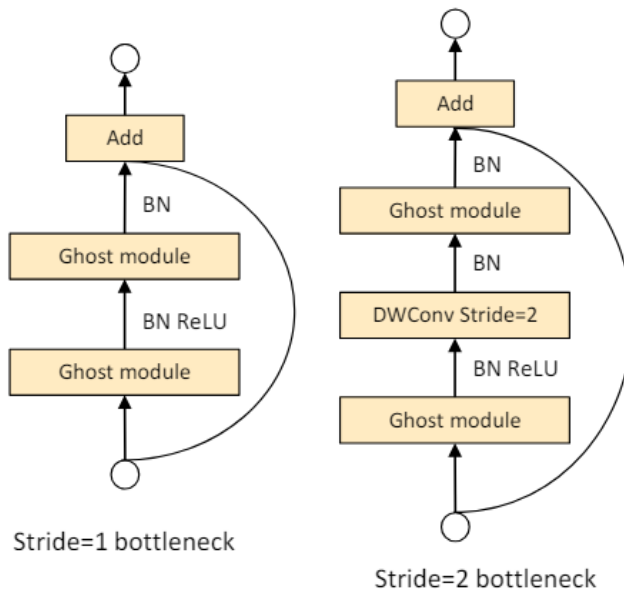


Figure 2.4: Ghost Bottleneck [WL⁺22]

Bottlenecks

The GhostNet, original network where GhostModules were used, was built using multiple GhostBottlenecks [WL⁺22]. The bottlenecks are of two types. Both of them start with a Ghost Module followed by a Batch Normalization with ReLU activation function. After that, the second type, with stride equal with 2 instead of 1, continues with a DepthWise convolution using the set stride, followed by a Batch Normalization. In both bottlenecks, a Ghost Module follows, with Batch Normalization. After the last mentioned module, the content brought by a skip connection is added to the current result. The architecture of the Ghost Bottleneck is presented in Figure 2.4

2.3.4 Attention Mechanisms

Classical CNNs need to make a lot of operations to learn the dependencies between two positions in an image [VSP⁺23]. This problem was solved by using attention mechanisms, focusing more on the most relevant parts of the photo, as a human, managing to obtain superior performances. Also, the model is adjusting the weights in order to decide how much attention each feature needs.

Attention Function

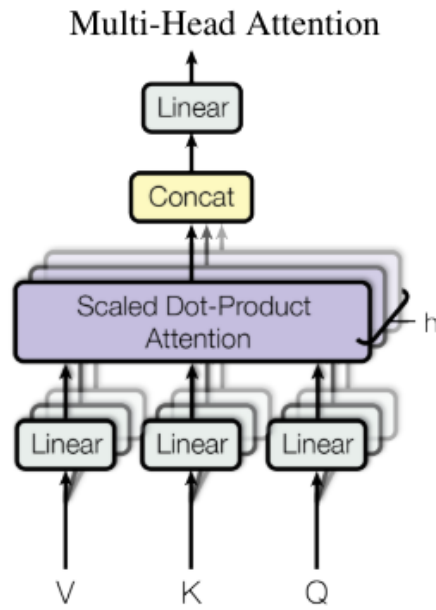
An attention function may be represented as a function $Y = f(Q, K, V)$, where Q , K and V are all vectors. f is the weighted sum of the values, where the w^i is computed using a function h , where h is the compatibility function between q^i and the corresponding key [VSP⁺23].

Scaled Dot-Product Attention

Scaled Dot-Product is one of the most commonly used attention function, together with additive attention [VSP⁺23]. The input of the function is formed by queries and keys, with dimension d_k , and values, with dimension d_v . The output of the function, Y is the dot product of the queries with all keys, all divided by divided by the $\sqrt{d_k}$. The weights of the values are obtained applying a softmax function.

Multiple Attention Heads

In the case of the Transformer model, the Attention Module repeats its computations in parallel through an Attention Head [VSP⁺23]. The Attention Module divides its parameters in n -ways, where n is the number of the heads and then passes every

Figure 2.5: Multi-Head Attention[VSP⁺23]

set to an Attention Head. Every Attention Head performs attention separately, allowing the model to capture different spatial relations in the input. After each head computes its output, the results are concatenated and linearly transformed into the final output, as shown in Figure 2.5, where h is the number of heads.

Chapter 3

Related work

The challenge of enhancing low-resolution images has long been a focus of research. With rapid technological advancements, there is an increasing proliferation of high-quality screens boasting large resolutions and a diverse color spectrum, along with the launch of high-resolution cameras each year. As a result, there is growing interest in revitalizing old photographs to meet contemporary standards of image quality. Also, as long as the clarity of the modern photos is not determined only by the sensors used, but at least equally by the processing algorithms used, the development of the SR models plays a crucial role in the race for the best photos. Over time, the solutions to enhance image quality have evolved into two primary categories: "traditional" deep learning super-resolution techniques and methods based on GANs.

3.1 Classical Deep Learning-based Methods

The super resolution algorithms have been discussed even before the 2000s, with the classical algorithms like Bilinear Interpolation, Nearest Neighbor Interpolation, Bicubic Interpolation, Maximum a posteriori, etc [AML24]. Starting from them, the task of Super Resolution has seen a huge evolution in the quality of the generated images with the usage of Convolutional Neural Networks in models like SRCNN and RCAN [DLHT15] [ZLL⁺18].

3.1.1 Super-Resolution Convolutional Neural Network

Super-Resolution Convolutional Neural Network (SRCNN) is one of the first proposed models intended to increase the resolution of the pictures [REKL18]. It introduces the powerful convolutional neural networks into the world of image resolution enhancement, achieving superior accuracy compared with state-of-art example-based methods which were previously used.

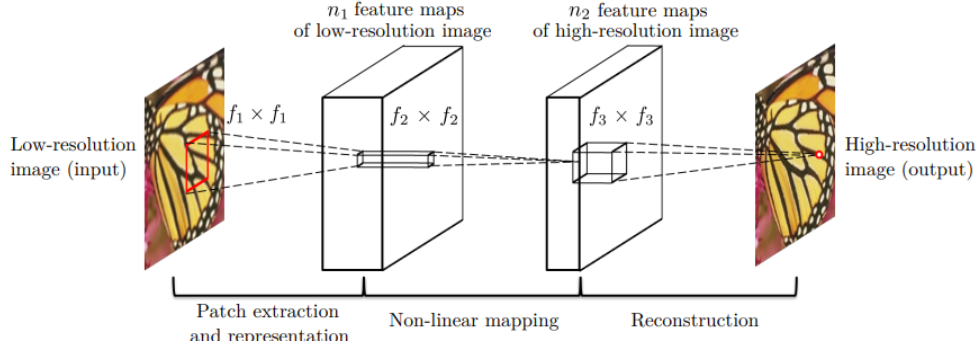


Figure 3.1: SRCNN Architecture [DLHT15]

SRCNN was designed with simplicity in mind [DLHT15]. This straightforward network comprises just three processing layers, as illustrated in Figure 3.1. Initially, the model upscales the low-resolution image to the desired high-resolution size using bicubic interpolation. The first layer is dedicated to patch extraction and representation, where the image is divided into overlapping patches to capture local features, aiding in the mapping to high-resolution counterparts. These patches, smaller than the full image, are represented as high-dimensional vectors. The second layer handles non-linear mapping, processing features from the first layer through one or more non-linear layers to learn complex patterns in the data. Although adding more layers can increase non-linearity, it also results in longer training times. The final layer focuses on image reconstruction. Unlike prior methods that average predicted overlapping high-resolution patches to form the final image, this model uses a convolutional layer to learn the reconstruction process, representing a significant innovation.

The network is trained using Mean Squared Error(MSE) between the set of high-resolution images and the set of low-resolution images and Peak signal-to-noise ratio is used for the evaluation of the result [REKL18].

This model's notable feature lies in its ability to achieve rapid processing suitable for real-time online applications, accomplished through a concise design with a limited number of filters and layers [DLHT15].

SRCNN presents a fully convolutional neural network, which learns directly the mapping between the high-resolution and low-resolution images and proved that deep learning approaches are very useful in computer vision problem of super-resolution, achieving good quality and speed [REKL17].

Besides of the benefits enumerated above, there is one significant limitation of that model. Being a PSNR-based model, it optimize the pixel difference between predicted and ground-truth high-resolution images [LTH⁺17]. Therefore, it will fail to reproduce realistic textures.

3.1.2 Enhanced Deep Residual Network

ESDR is a classical deep learning-based model that has achieved superior results by bringing multiple innovations on the table [LSK⁺17]. The authors studied existing models at that time, concluding that most of them are very sensitive regarding minor architectural changes. They analyzed the common problems between the existing models, such as dealing with various in a complex manner.

The authors started from SRResNet and developed the presented one [LSK⁺17]. In order to achieve good results using a model that was previously used for problems such as image classification and detection problems, they analyzed and removed some unnecessary modules with the purpose of simplifying the network. A second step was to investigate methods to share the parameters between different scales.

Another impactful change reported to the SRResNet was the change of the Residual Block structure [JZZ⁺21]. The authors decided to remove the Batch Normalization layers, resulting in a lighter block, used to build a larger model, with better performance.

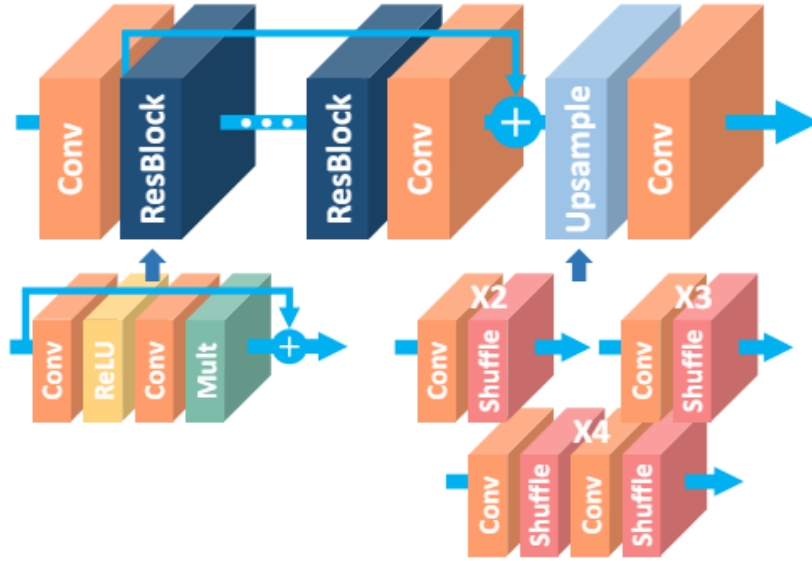


Figure 3.2: ESDR Architecture [LSK⁺17]

The network architecture is divided into two main components: downsampling and upsampling [KL18]. The upsampling section includes a Convolutional Layer followed by several Residual Blocks. Each Residual Block is composed of a Convolutional Layer, a ReLU activation layer, another Convolutional Layer, and a final layer that adds the output of the last convolution to the result from a skip connection between the Addition Layer and the input. The output from the last residual block is filtered through a Convolutional Layer and then passed through an Addition Layer,

which incorporates the output of the initial Convolutional Layer. The upsampling component utilizes Convolutional Layers and Shuffle Layers. The model's architecture is depicted in Figure 3.2.

The model was trained for upsampling with factors $\times 3$ and $\times 4$ using transfer learning from $\times 2$ network, accelerating the training and improving the quality of the generated images.

3.2 GAN-based methods

While the classical deep learning methods are producing very qualitative high-resolution images, with faster and faster algorithms, that are producing better results from the perspective of PSNR, there is a major downside, failing to reproduce realistic textures [LTH⁺17]. The classical loss functions do not take into consideration the importance of high-frequency details, producing results that are easily distinguished by the human eyes from the real ones. The usage of GAN-based architectures has proven to have way better results.

3.2.1 Super-resolution Generative Adversarial Network

Despite the breakthroughs discoveries regarding the super-resolution problem, with many models that are achieving important milestones in accuracy and speed, using faster and deeper CNNs, there is still a huge problem that needs to be solved: the state of art models are losing high-frequency textures [LTH⁺17]. In other words, the generated images tend to be blurry, being a little bit low quality pictures from the perception of the human eyes.

SRGAN, the first framework capable of inferring photo-realistic natural images, is developed by analyzing existing models and bringing a new direction to the table. One of the most significant changes is the proposal of the perception loss function [LTH⁺17]. Inspired by humans' behavior of extracting and comparing features, the perceptual loss function aims to do the same. Unlike the calculation of the loss function considering only the generated pixel values and the ground truth values, the discussed loss function calculates the loss between the simulated and the real values at the feature level. The basis of the perceptual loss is a pre-trained feature extractor, for example VGGs [WK22] [SZ14]. SRGAN uses a loss function consisting into an adversarial loss and a content loss. The purpose of the adversarial loss is to keep the solution into the direction of natural images, using a discriminator network. Content loss is improving the overall result by keeping an eye on the perceptual loss.

The aim of the SISR is to estimate high-resolution images I^{SR} from the low-resolution images I^{HR} [DLHT15]. The proposed network is trained downsampling

high-resolution images, applying a Gaussian filter followed by a downsampling. The purpose of the training is obtain a G function, responsible for generating the corresponding I^{HR} from a I^{SR} . In the case of the discussed model, the generator is a feed-forward CNN G_{θ_g} , parametrized by θ_g . Here θ_g denotes the weights and biases of a L -layer deep network. For training images, we solve:

$$\hat{\theta}_G = \arg \min_{\theta_G} \frac{1}{N} \sum_{n=1}^N \mathcal{L}^{SR} (G(I_n^{LR}), I_n^{HR}) \quad (3.1)$$

Furthermore, in the case of SRCNN, there is a discriminator defined following the Goodfellow et al. [?]. The generator is trained with the purpose of fooling the discriminator. The generator network consists in B residual blocks, with identical layout. The residual blocks contains 2 convolutional layers, 2 batch normalization layers, a ParametricReLU layer for the activation function and an Elementwise Sum layer. The arhitecture of the dicriminator uses LeakyReLU as the activation function and avoids max-pooling. The discriminator contains 8 convolution layers with a number of 3×3 filter kernels, doubling the number of kernels in the VGG network. The result of the network is passed through 2 dense layers and a final sigmoid activation.

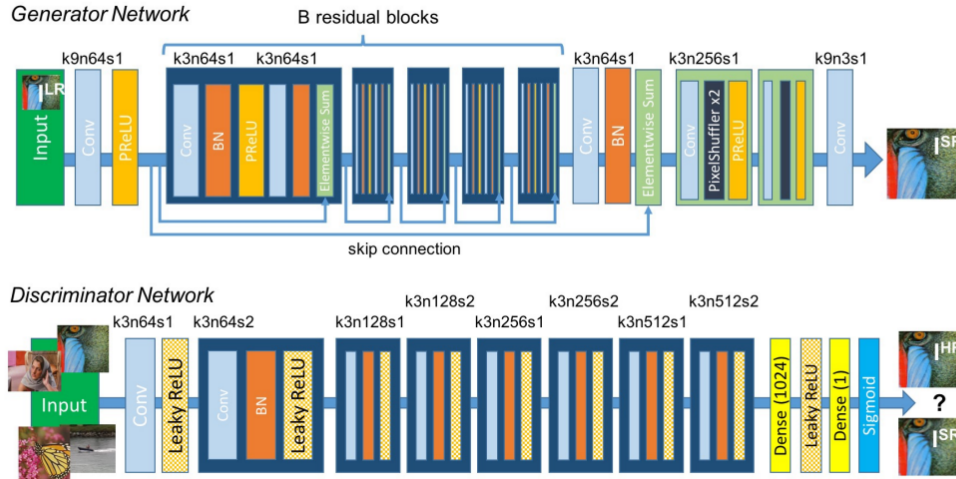


Figure 3.3: SRGAN Arhitecture [LTH⁺17]

One of the key aspects of that SRGAN is using the perceptual loss. The authors defines that loss as the weighted sum of a content loss and an adversarial loss component, as described by the following formula:

$$l^{SR} = \underbrace{l_X^{SR}}_{\text{content loss}} + \underbrace{10^{-3} l_{Gen}^{SR}}_{\text{adversarial loss}} \quad \text{perceptual loss (for VGG based content losses)} \quad (3.2)$$

The authors used pixel-wise MSE loss computed between the high-resolution ground truth image and the inferred one as the content loss component.

The adversarial loss is determined by the likelihood that the discriminator will accurately classify an image.

As we can observe in the following in the Figure 3.4, the SRGAN is generating superior results compared with the classical mathematical methods and with the classical deep learning methods.

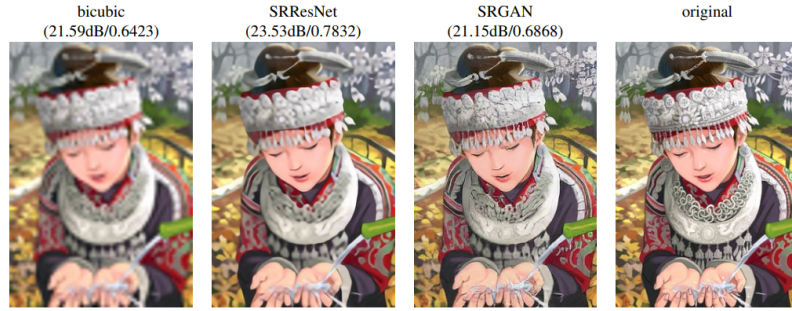


Figure 3.4: Comparative results of the SRGAN model. Corresponding PSNR and SSIM are shown in brackets [LTH⁺17]

3.2.2 EnhanceNet

EnhanceNet uses a fully convolutional network (FCN) with residual learning, integrating an additional component into the loss function to understand finer texture details. Beyond the lossy highlights in SRGAN, it incorporates a texture loss similar to Style Transfer, enhancing its ability to capture elaborate texture information, resulting in very qualitative results, as shown in Figure 3.6 [SSH17].

The using of only convolutional layers allow training only a single model for all low resolution, regardless of size, as shown in Figure 3.5 [SSH17]. The model is using only 3x3 filters, being inspired by the VGG architecture.

Given that the low-resolution (LR) input image is smaller than the target high-resolution output, it needs to be upscaled during the process to produce a high-resolution estimate [SSH17]. Although one might think of using bicubic interpolation to enlarge the LR image before feeding it into the network, this method introduces unnecessary redundancy and increases computational load. Instead, convolutional neural networks (CNNs) use convolution transpose layers to manage up-sampling of feature activations directly. This technique eliminates the need to input a larger, redundant image into the CNN, allowing the majority of computations to occur within the LR image space. Consequently, this results in a more compact network and enables filters to have larger receptive fields in relation to the output

Output size	Layer
$w \times h \times c$	Input I_{LR}
$w \times h \times 64$	Conv, ReLU
	Residual: Conv, ReLU, Conv
	...
$2w \times 2h \times 64$	2x nearest neighbor upsampling Conv, ReLU
$4w \times 4h \times 64$	2x nearest neighbor upsampling Conv, ReLU
	Conv, ReLU
	Conv
$4w \times 4h \times c$	Residual image I_{res}
	Output $I_{est} = I_{bicubic} + I_{res}$

Figure 3.5: EnhanceNet Architecture [SSH17]

image. However, it has been observed that convolution transpose layers can sometimes generate undesirable patterns in the output. To address this, an additional regularization term is introduced to the model. Moreover, to further minimize artifacts, a convolution layer is added after each upsampling block.

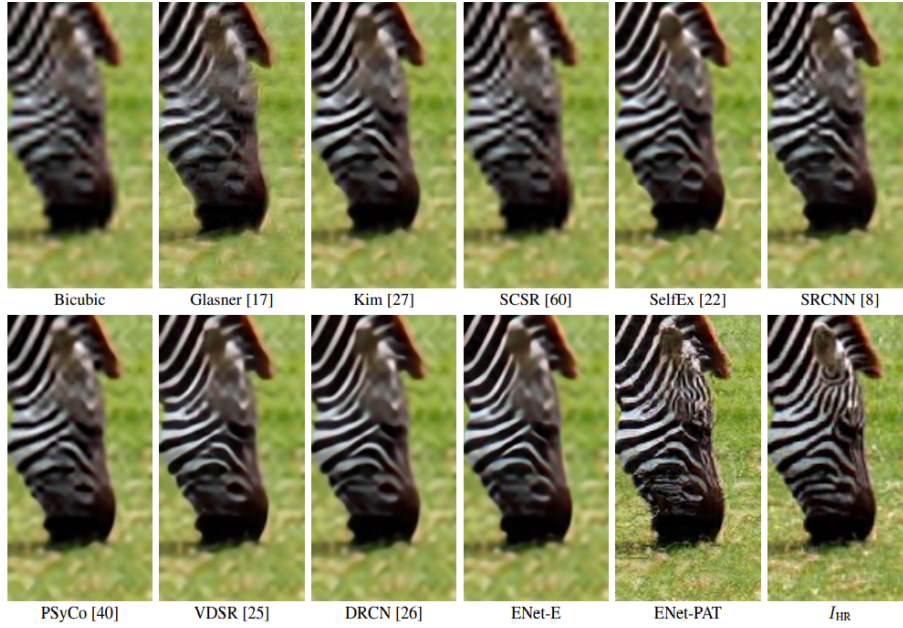


Figure 3.6: EnhanceNet Comparison [SSH17]

A key innovation introduced by EnhanceNet is the replacement of stacked convolution layers with residual blocks, which leads to faster convergence [ZZZ⁺19]. Additionally, incorporating the bicubic interpolation of the input into the model's output helps stabilize the training process and minimizes color shifts in the output during training.

EnhanceNet employs adversarial training, a powerful technique for generating images with realistic appearance [ZZZ⁺19]. This method involves a competition

between two neural networks: the generator and the discriminator. The generator creates synthetic data that mimic the real data from the training set, while the discriminator functions as a critic, distinguishing between genuine samples from the training set and the synthetic ones produced by the generator. The model monitors the discriminator's average performance and trains it further only if its accuracy in distinguishing between real and fake data falls below a certain threshold.

3.2.3 Enhanced Super-Resolution Generative Adversarial Networks

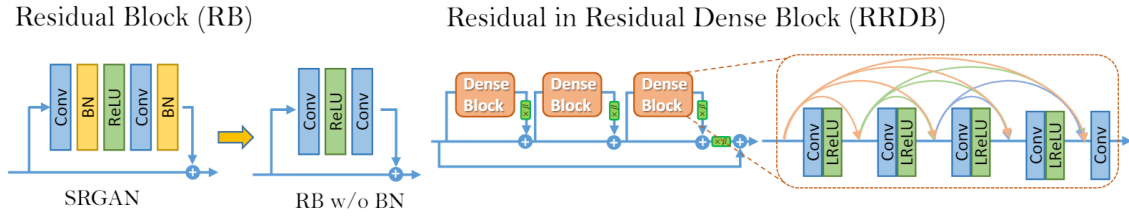
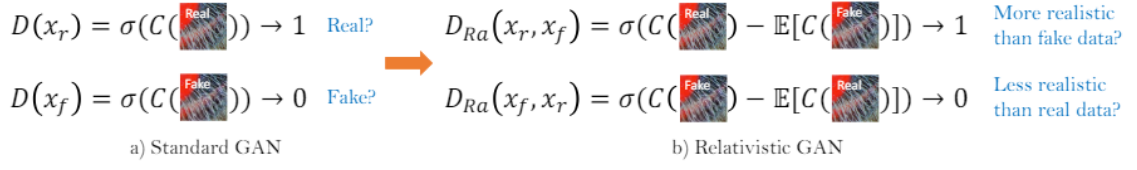
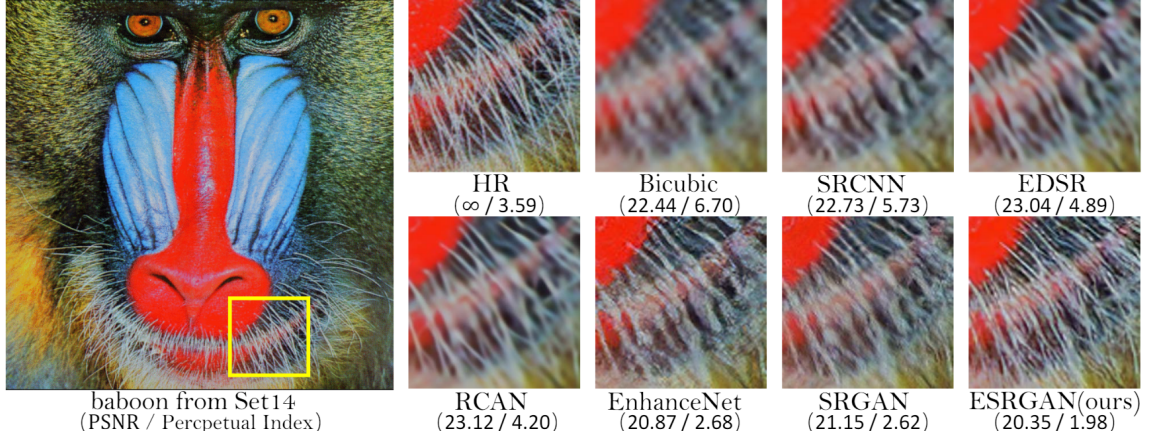


Figure 3.7: Residual in Residual Dense Block [WYW⁺18]

The authors of that model highlighted that SRGAN has a very good performance, producing excellent results from the perception of the human eyes, but there are still a lot of hallucinations in the generated images [WYW⁺18]. The ESRGAN is made starting from the SRGAN, but addresses its problems changing the network architecture and improving the loss functions. The model introduces Residual in Residual Dense Block (RRDB) as the basic building block, shown in Figure 3.7, and drops the batch normalization. Moreover, a key of innovation is the usage of a relativistic GAN. Now, the discriminator is not only predicting whether an image is real or fake, it is also computing the similarity between a fake and a real one. The perceptual loss function is improved by using features before activation. Using an activation function leads to the loss of information because the function can clip the data or alter the values. Therefore, the perceptive loss function used in the ESRGAN provides a better focus for brightness and textures. The Figure 3.9 show the results of ESRGAN image enhancing on the baboon photo from Set14, compared with other methods.

Relativistic Discriminator

The main difference from the classical discriminator is the estimation of probability that a real image x_r is more realistic than a fake one x_f , as shown in the Figure 3.8, where $C(x)$ is non-transformed discriminator output and $\mathbb{E}$ is the operation of taking average for all fake data in a mini-batch. The


 Figure 3.8: Standard and Relativistic Discriminator [WYW⁺18]

 Figure 3.9: ESRGAN Comparison [WYW⁺18]

3.3 Conclusion

As shown above, the Super Resolution problem is a task that has received a lot of attention, having multiple applications. There are more and more models every year that are further improving the quality of the inferred images.

As we can see in the above sections, the evolution of the SR models was made step by step, from the classical algorithms to the classical deep-learning based methods and from classical deep-learning based methods to the GAN-based models. Every new model proposed more or less impactful changes, but all brought significant discoveries in this field.

As a summary, the most game-changing techniques used were:

1. the usage of Convolutional Neural Networks
2. the usage of the Residual Blocks, who has proven to optimize the models pretty well from the perspective of both the quality of the results and the usage of the resources needed to train
3. the usage of the GAN-based architectures, reducing the loss of the textures
4. the usage of the perceptual loss function, that has led to the decreasing of the lost high-frequency features

5. the usage of the Residual in Residual Dense Block
6. the usage of the relativistic discriminator

We have presented a brief review of the deep learning models that had a consistent impact at the time of their proposal. We reviewed the new paradigms brought by SRCNN, ESDR, SRGAN, EnhanceNet and highlighted the added quality of the ESRGAN.

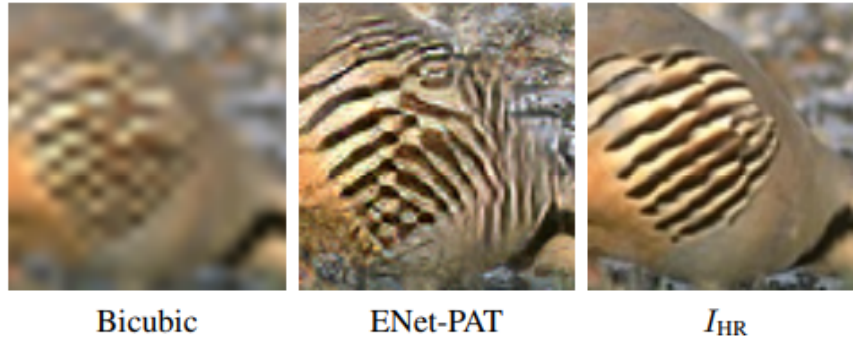


Figure 3.10: Failure case of EnhanceNet [SSH17]

Even if the GAN models are being focused on the real textures, emphasizing the limitations of PSNR-focused images, there are still interesting failures on some images [LTH⁺17] [SSH17]. A pertinent example is on the Figure 3.10, where the model continues a pattern visible in the LR image onto smooth areas. This is an example of model hallucinating textures.

While the presented models are already competitive in terms of their runtime performance on high-resolution data, future work may decrease the depth of the network using higher performing levels or maybe improve the used architectures and the loss functions. Also, those types of innovations may lead to a reduced number of hallucinations and even more realistic textures. Another key aspect regarding the possible improvement of SR models may be using naturally degraded images instead of artificially generated ones.

Chapter 4

Ghost Enhancer Mobile Application

In this section we will focus on the the architecture implementations details of our model.

4.1 Super Resolution Enhancer

In this section, we present the design and implementation of our Super Resolution deep learning models, GhostSRGAN and SA GhostSRGAN. The models are made starting from one of the most powerfull Super Resolution State-of-Art model: SRGAN.

4.1.1 Model Architecture

Generator Architecture

As we discussed above, GhostSRGAN and SA GhostSRGAN are designed starting from SRGAN. One of the most major improvements at the level of the generator is the substitution of the Residual Blocks with Ghost Bottlenecks, as shown in in Figure 4.1, reducing the computational complexity of the model because of creating a more compact model without sacrificing performance.

In the case of the SA GhostSRGANm, after each Ghost Bottleneck, we add a Multi-head layer to improve the quality of the generated images capturing the dependencies at the level of the pixels for each feature. In order to reduce the risk of overfitting, we use Multi-Head Attention Layers, aggregating the information from multiple perspectives in order to infer the result.

Viewed as a whole, the network begins with a Convolutional Layer followed by a Parametric Rectified Linear Unit layer. This layer applies a convolution with a kernel equal with 9, having as purpose to extract the features from the input image, having as a result a number of 64 channels. After that, we continue with the

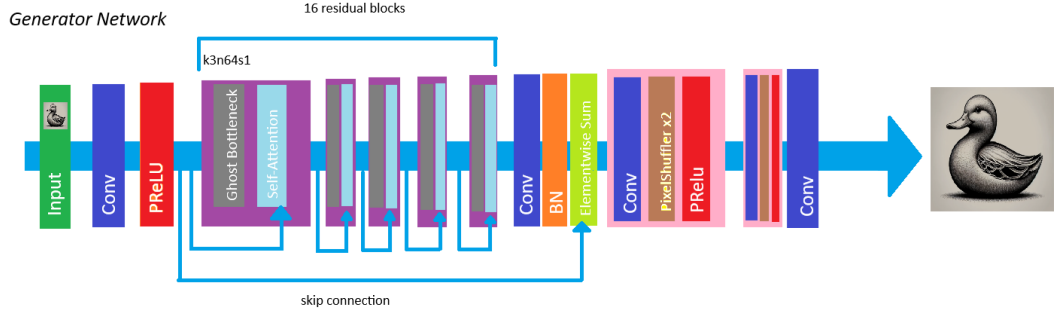


Figure 4.1: SA GhostSRGAN

Ghost Bottlenecks and Multi-head Attention Layers in the case of the Self-Attention model, having also as a result a number of 64 channels. There are skip connections to every Attention Layer. After the last Self-Attention Layer, there are a Convolutional Layer, a Batch Normalization and an Elementwise Sum layers. After that, the upsampling part begins. It consists in 2 passes through a Convolutional Layer, PixelShuffle layer and a Parametric Rectified Linear Unit layer. After that, a new convolution is applied to generate the final image.

Discriminator Architecture

The discriminator used in the case of the GhostSRGAN is the same as in the SRGAN. It has a first convolution with the kernel equal with three, followed by the Leaky ReLU activation function. After that, we have a number of seven basic blocks. Every basic block contains a convolution with the kernel equal with three and stride equal with two. A dense layer is collecting the information from the last basic block, followed by applying of Leaky ReLU. The output is forwarded to another Dense Layer, followed by applying the Sigmoid function.

The discriminator used in the case of the SA GhostSRGAN is created starting from the GhostSRGAN version, but it has a Multi-head Attention Layer after every Basic Block.

4.1.2 Dataset

In order to obtain a model that performs as well as possible on various images, with various textures and other elements, we trained our model on various dataset. Since, generally speaking, deep learning models are pushed more to their limits by the images with human faces rather than anything else, we focused more on that type of photos at the creation of the dataset.

Our dataset was created using 202599 images with faces from the CelebA [LLWT15]. Moreover, we added 10324 images with outdoor scenes, from SFTGAN dataset [WYDL18]. We also used a subset of 1600 images from Flickr 30k. Every image was resized to dimensions of 128x128x3. The images were shuffled and grouped in folders with the size 1000.

The low-resolution images were created applying bicubic interpolation on the original images and reducing their dimensions to 32x32x3.

In order to compare our models with the state-of-art methods, we took 61k random images from the CelebA dataset and trained both our models and compared models.

4.1.3 Training and deployment

The model was implemented and trained using PyTorch. For the comparison with previous models, we trained both GhostSRGAN and SA GhostSRGAN and the classic models on RTX 3050 6GB Laptop GPU and let the model run for 32 epochs. The batch size was set to 32. For our models, we used a constant learning rate of 8×10^{-5} . We used Adam Optimization Algorithm for the generators and for the discriminators. The betas were set to $b1 = 0.5$ and to $b2 = 0.999$. For the complete training of the GhostSRGAN, we used the same hyper-parameters, but we let the model run for another 68 epochs.

4.1.4 Experimental Results

Table 4.1 provides sample inferences of our models compared with the related works and the target image (Ground Truth). As we can see, GhostSRGAN and SA GhostSRGAN are generating images with a quality comparable with the state-of-art methods.

Because nowadays, more and more models are intended to be used on mobile devices or in real time applications, the speed of the processing is a very important factor. With our models, we mainly focused on decreasing the necessary time for inferring multiple photos. In the Table 4.2 we presented the necessary running time required by our models compared with the existing ones.

For a better understanding of the performance of the compared models, we computed a series of metrics, presented in the Table 4.3. We used PSNR, SSIM and MAE as metrics.

Table 4.1: Models results

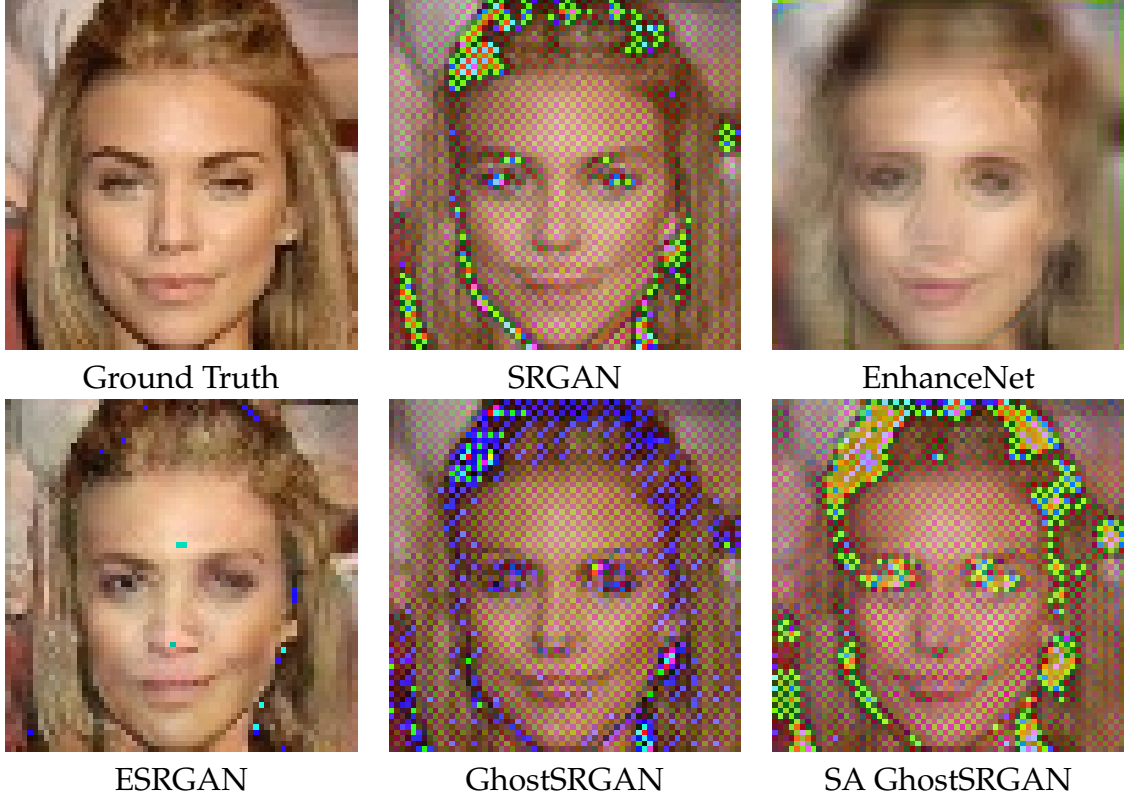


Table 4.2: Running time for 5000 images

Model	Time
SRGAN	10.98s
EnhanceNet	8.79s
ESRGAN	51.06s
GhostSRGAN	5.27s
SA GhostSRGAN	9.85s

Peak Signal To Noise Ratio (PSNR)

PSNR, or Peak Signal-to-Noise Ratio, measures the relationship between the maximum potential power of a signal and the power of corrupting noise that impacts the clarity of an image [SAU19]. A higher PSNR value indicates superior image quality. The calculation of PSNR is performed using a specific formula:

$$\text{PSNR} = 10 \log_{10} \left(\frac{\text{peakval}^2}{\text{MSE}} \right) \quad (4.1)$$

where *peakval* represents the maximum value of a pixel. *MSE*, Mean Squared Error, measures the average of the squares of the errors, using the Formula:

$$\text{MSE} = \frac{1}{MN} \sum_{n=0}^M \sum_{m=1}^N \left[G(n, m) - \hat{G}(n, m) \right]^2 \quad (4.2)$$

Table 4.3: Performance indexes

Model	PSNR	SSSIM	MAE
SRGAN	10.73	0.23	0.17
EnhanceNet	17.02	0.57	0.08
ESRGAN	16.68	0.55	0.06
GhostSRGAN	10.20	0.23	0.17
SA GhostSRGAN	10.24	0.13	0.22

where $\hat{G}$ is the predicted value and G is the value we observe, M and N are the sizes of the image.

Structure Similarity Index Method (SSIM)

SSIM, or Structural Similarity Index Measure, is a model based on human perception [SAU19]. It evaluates image quality by measuring the perceptual change in visual information. This index assesses various elements that are readily noticeable to the human eye to deliver the most accurate assessment possible. Important factors are the luminance (μ_x, μ_y) , the contrast (σ_x, σ_y) and the structure (σ_{xy}) . In the context of the Super Resolution methods, its results are more relevant than those given by the PSNR. SSIM values range from -1 to 1, where 1 indicates perfect similarity. The SSIM is computed using the formula:

$$\text{SSIM}(x, y) = \frac{(2\mu_x\mu_y + c_1)(2\sigma_{xy} + c_2)}{(\mu_x^2 + \mu_y^2 + c_1)(\sigma_x^2 + \sigma_y^2 + c_2)} \quad (4.3)$$

where c_1 and c_2 are two constants used to stabilize the division.

Mean Absolute Error (MAE)

MAE represents the mean error is an error metric that calculates the absolute average of the pixels between the real image, I_r , and the generated one, I_f , computed using the formula:

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i| \quad (4.4)$$

where N is the total number of pixels and $\hat{y}_i$ and y_i are the pixels of the reconstructed and original images. A smaller MAE means that the generated image is closer to the real one.

4.1.5 SWOT Analysis

In this part we perform a SWOT analysis of the proposed solution for image super resolution model. This part wants to identify the strengths and weaknesses of the model, together with some concerns and limitations.

Strengths

1. **Portability:** The usage of Ghost Bottlenecks instead of the classical residual blocks resulted in a huge improvement from the point of processing speed, making the model to be very easily run on every type of mobile devices.
2. **Good Accuracy:** Our model was developed starting from the SRGAN and adding some impactfull changes, resulting in a model that is keeping the quality of the State-of-Art models.
3. **Adaptability:** The model has proved to be very efficient in the super resolution problem, being able to be used in a various numbers of tasks, from image enhancing to surveillance.

Weaknesses

1. **Training Data Dependency:** The model needs a lot of data to be trained, like others deep learning models. The data diversity during training is influencing the results of the real world running.
2. **Hallucinations:** There are still a lot of hallucinations in some special cases, especially regarding the textures and the background from the images with faces.
3. **Texture Generations:** The generated textures are not yet very veridical in some cases.
4. **Heavy training:** Even if the model runs very quickly on mobile devices, the training requires a huge number of images and it is very resource-intensive from the point of view of computation.

Possible improvements

1. **Quality Optimizations:** Some Attention Mechanisms proved to be very effective in image generation problems. There is a potential improvement using different network architectures and different Attention Layers.

2. **Integration With Other Image Enhancing Techniques:** There are some models that can be integrated with the presented ones, resulting in full image recovery.
3. **Live processing:** Due to the low timestamp required to process huge batches of images, the presented models could be used for live enhancing.

Ethical Concerns

1. **Privacy Concerns:** Enhancing images can lead to the revealing of some unseen details, putting the initial owner or the actors from the photo in a bad light.
2. **Hallucinations problem:** The hallucinations that may appear in the generated images can lead to the exposure of some artifacts that seem to be something else, resulting in fake images.

4.2 Mobile Application

In this section we will present the mobile application, designed to enhance images. We named it Ghost Enhancer because of the neural network used to increase the resolution of the images.

4.2.1 Technologies

The application is made in Swift, targeting all of the Apple iPhone mobile phones that are running the 17-th version of the iOS. During the development of the product, we used the latest technologies available at the time, creating a native application, respecting the best code guidelines presented by the Swift developers.

Xcode

Being probably the most popular IDE for Swift development, Xcode is a modern solution used for the crafting of our mobile application. It offers accurate previews for the user interface components, efficient code autocomplete and it is designed to be able to connect the phone and load the application without too much effort. It also offers simulators for a wide range of devices that are targeted for Ghost Enhancer.

SwiftUI

One of the ingredients of the code's modernity was the use of the SwiftUI framework, increasing the percentage of the functional programming part involved in the

project, resulting in a more scalable solution, with a cleaner code, also reducing the development time.

SwiftData

The usage of the SwiftUI unlocked the integration of a very versatile and robust solution for the data persistence: SwiftData. That solution fits very well on our needs, being very efficient for the store of images, requiring a very few lines of boilerplate code and less development time compared with previous frameworks.

CoreML

```
import coremltools as ct

model = ct.convert(
    traced_model,
    inputs=[ct.ImageType(shape=(1, 3, 16, 16))],
    outputs=[ct.ImageType()],
    minimum_deployment_target=ct.target.iOS17,
)

# Save as .mlpackage since .mlmodel is not supported for this model
model.save("IE1.mlpackage")
```

Figure 4.2: CoreML Converter Tool

One of the best machine-learning frameworks, engineered to optimize the performance of the models on iOS, MacOS and others operating systems from the Apple Ecosystem, CoreML has facilitated the usage of our model, built using PyTorch, in a very scalable and easy to use way. Being a native solution, it has a good integration in Xcode, offering even previews of the model, allowing us to see if the model is working properly.

The python model was converted using the tool offered by CoreML, using the code from the Figure 4.2.

SF Symbols

The focus of the current application was to prove the performances of the GhostSR-GAN and to offer a user-friendly user experience. Without an intuitive design we would not have been able to reach these goals. Beside having intuitive organization between different flows of the application, we needed good icons, so here is the part where SF Symbols shined.

4.2.2 Application Architecture

In the development phase of the application, we tried to use a clean architecture in order to obtain a mobile application that is as easy as possible to be scaled. There is a huge potential to add more features and it has to be made as easy as possible. The application is separated in different independent layers in order to facilitate the further development.

Domain Layer

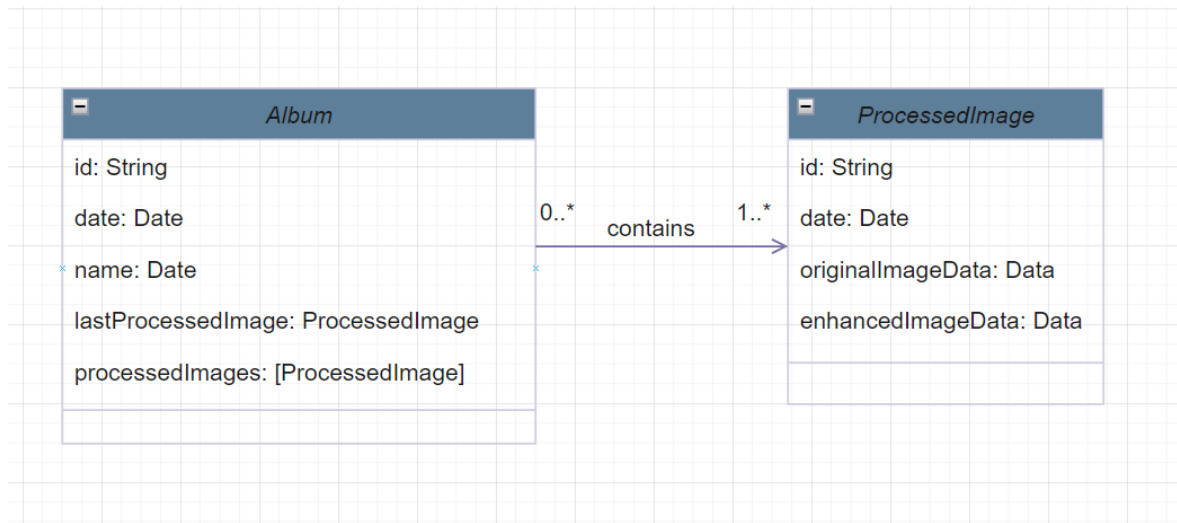


Figure 4.3: Model Class Diagram

Our application is based on two entities that are persisted between different usage sessions: `ProcessedImage` and `Album`. Our application let the user to create multiple albums with multiple enhanced images and every album will contain multiple instances of the `ProcessedImage` entity, as shown in Figure 4.3. Every album has an id, a name that can be changed by the user during the usage of the application, the creation date, last processed image and a list of all processed images. The `ProcessedImage` entity has an id, the processing date, the data of the original image and the data of the enhanced image.

In the domain layer, we also have a component name `SuperResolutionService` that has the super resolution model. It is loading the model at the initialization time and has a function that is returning the enhanced image applying the GhostSRGAN model:

```
func analyzeImage(image: UIImage) async -> UIImage?
```

Data Layer

The data layer contains only the persistence part needed for the Album and for the ProcessedImage and it is managed by the SwiftData.

Presentation Layer

The presentation layer is responsible for the interaction between the user and the application, being more complex than the other layers.

It has a TabController with a TabControllerViewModel. The TabController redirects the user to one from the following pages: HomeView, CameraView and GalleryView.

The GalleryView is the default opened tab. It conforms to the View protocol from SwiftUI, containing a ViewModel, the body and the displayed views. The displayed views are:

1. **Header:** The name of the application.
2. **Albums:** The albums of the user. That component contains the last album, displayed at the top and a grid with the other albums. The albums are shown using AlbumCard, a view that deals with the graphic representation of the albums. The user can open a CameraView for the current album tapping on the AlbumCard.

The CameraView that can be opened from the TabController creates a new album. Every CameraView conforms to the View protocol from SwiftUI and has a ViewModel that handles the business logic for the view. The user can open the AccessCameraView from here or the UploadPhoto. The user can see the list of the enhanced images for that album.

The GalleryView conforms to the View protocol and has a ViewModel and a body computed variable. The body has a header and a grid with all of the processed images of the user.

Tapping on an image in any part of the application leads to the opening of the PhotoViewer component. The user can zoom in or zoom out the photo, delete the photo, see both the enhanced and the original images, export them.

The architecture of the Presentation Layer is presented in Figure 4.16.

Extensions

There is a long list of benefits that determined us to use Swift extensions as much as possible:

1. **Cleaner Code:** The code is organized in a more logical way, making it more understandable by other persons.

2. **Modernity:** There is a strong focus on the declarative programming nowadays. In some cases, it leads to less code, reducing the development needed time. The usage of the extensions enable us to turn our face to this paradigm.
3. **Protocol Conformance:** One of the most powerful features of extensions is their ability to add protocol conformance to a class, structure, or enumeration. This means that you can declare the adoption of a protocol on a type separately from the type's main definition. Organizing your code in this way can lead to clearer and more modular code. For example, if a type needs to conform to multiple protocols, you can handle the implementation of each protocol in its own extension, keeping related methods grouped together and separate from the type's core functionality
4. **Adding Utility Functions:** Extensions can be used to add utility functions or computed properties to existing types, which can enhance their functionality without modifying the original type definition. This is particularly useful for adding custom methods to built-in types like String or Int. For example, you might add a custom extension to String to provide a shorthand for trimming whitespace, or extend Int with a property that returns the number as a formatted string.

Our most significant created extension is ProcessedImage. At the base, ProcessedImage is a persisted entity, containing the id, date, the data of the original image and the data of the enhanced image. In order to use that component in the Presentation Layer, we need to convert the data of the images into actual images. We can make it either by adding two computed variables in the Domain Layer, but that solution will destroy the logical separation of the layers, or we can make functions to do that where we need, resulting in duplicated code. We solved that problem by simply creating an extension in the Presentation Layer with two computed variables *enhancedImage* and *originalImage*.

Resources

Most of the mobile applications are delivered to a wide range of public. To offer an user experience as good as possible, the applications is recommended to have the texts in the language configurated on their phones. Our application only supports english at the moment, but we manage that by a class Strings that returns the displayed string for every part of the app. Currently, our class has multiple static constants, but it can be very modified to add multiple languages.

4.2.3 Application Flows

The application aims to offer the minimum functionalities to provide the most complete experience for users. We have designed a minimalistic user interface.

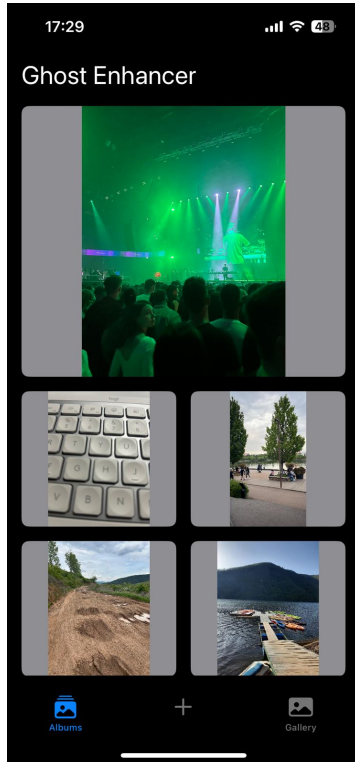


Figure 4.4: Home Screen

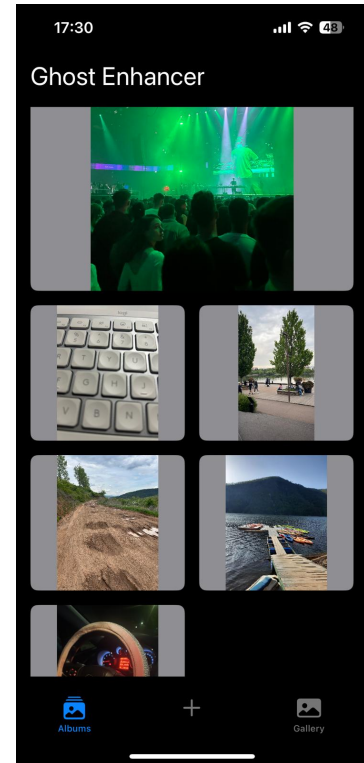


Figure 4.5: Home Screen Scrolled

Home Screen

The application opens in a HomeScreen, presented in the Figure 4.4 and in the Figure 4.5, with a list of all albums. The last created album is highlighted at the top of the screen and the rest of the albums are sorted descending by the creation date. The user can scroll the list of the albums. Every album presents the last enhanced image. If the user taps on an album, it is redirected to the Album Details page.

Album Details

The Album Details page presents a list of all the enhanced photos from an album. Starting from the design paradigm of the Home Screen, the last processed image is highlighted at the top of the screen, as shown in the Figure 4.6. Below that, it is a horizontal list with the other images, sorted descending by the processing date. The user can scroll the list to see the older photos, as shown in the Figure 4.7. By simply pressing the title of the album, the user can edit it.

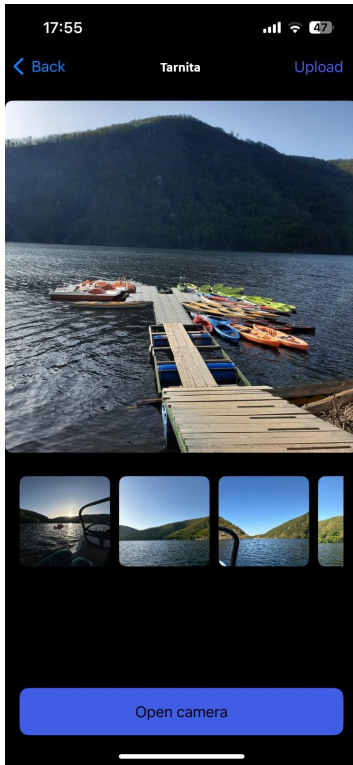


Figure 4.6: Tarnita De-
tails

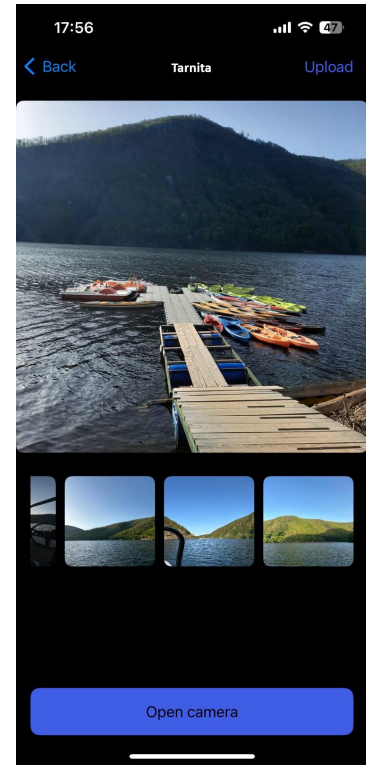


Figure 4.7: Tarnita De-
tails Scrolled

From that screen of the application, the user can add enhance more photos and add them to the current album.

First method, that will probably be the most used, is by uploading a photo from the user mobile phone, pressing the button from the right of the title, opening the screen shown in the Figure 4.8. The user can simply select a photo by pressing it.

Another method is to make a photo. As the iPhone cameras are producing very qualitative photos, the purpose of that flow is not to substitute the pre-installed camera application. That feature is intended for capturing a very old photo that is in physical format. The flow is shown in the Figure 4.9 and in the Figure 4.10.

By tapping on an image from the Album Details, the user is redirected to the PhotoViewer screen, shown in the Figure 4.13.

Gallery Screen

In this screen, the user can see all of the processed photos, as shown in the Figure 4.11. The photos are sorted descending by the date when they were processed. The user can scroll vertically to see the images that are not on the screen, as shown in the Figure 4.12. Every photos is tappable, redirecting the user to the PhotoViewer for that photo, presented in the Figure 4.13.

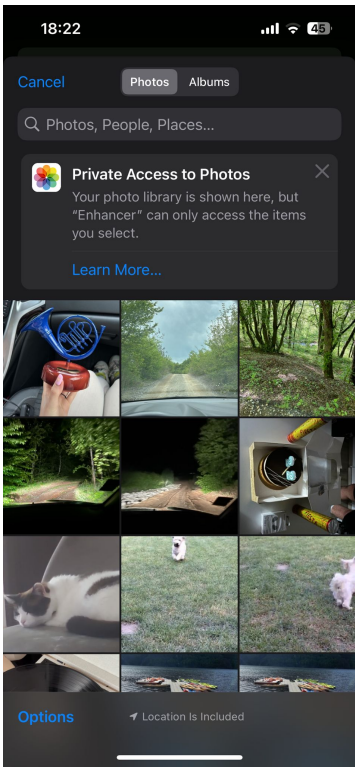


Figure 4.8: Upload photo

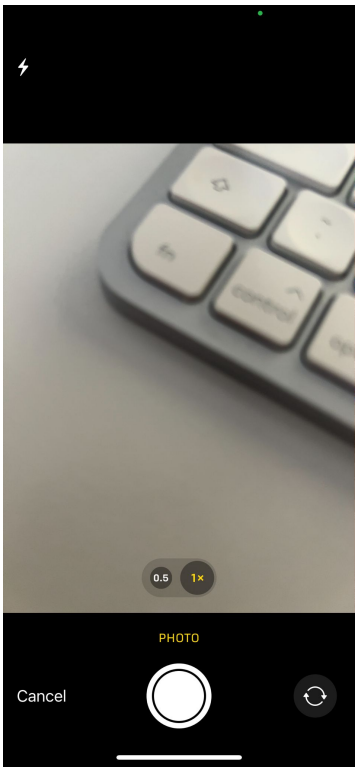


Figure 4.9: Make a photo

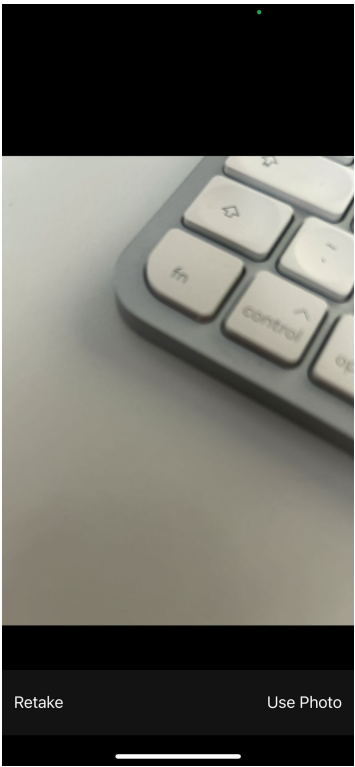


Figure 4.10: Use Photo or Retake

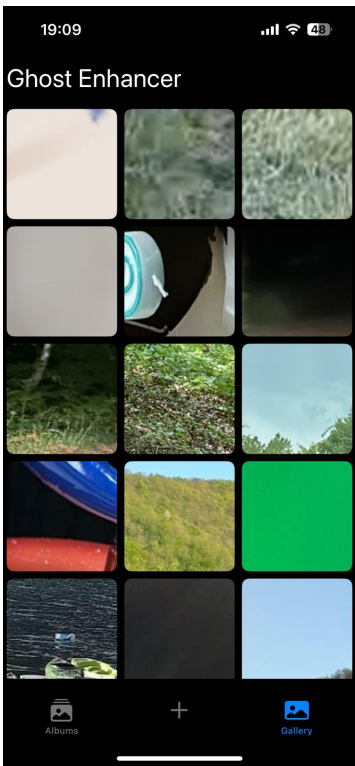


Figure 4.11: Gallery

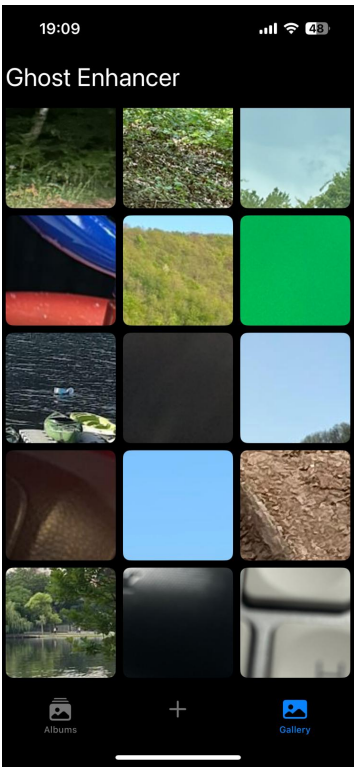


Figure 4.12: Gallery Scrolled



Figure 4.13: Photo Viewer Original

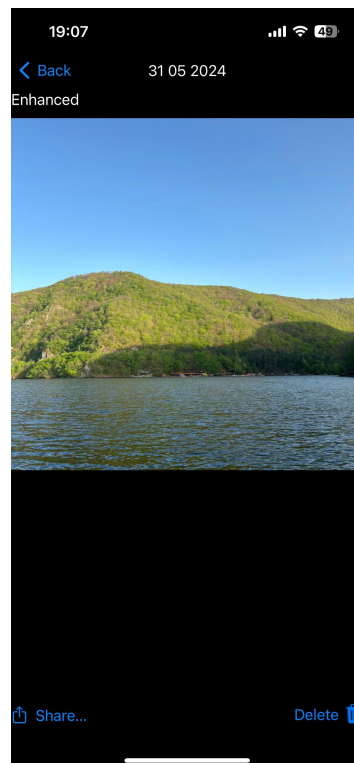


Figure 4.14: Photo Viewer Enhanced



Figure 4.15: Photo Viewer Zoom

Photo Viewer

This is the screen where the user can see the enhanced image and the original one, shown in the Figure 4.13 and in the Figure 4.14. The original photo has the label *Original* and the enhanced one has the label *Enhanced*. The user can see the date of the photo processing at the top of the screen. The user can share both the original and the enhanced images pressing the button from the left, in the bottom of the page. Also, the user can delete the photo pressing the button from the right, in the bottom of the page. That action will lead to the delete of the both the enhanced and the original image.

One of the major features of that page is photo zoom, shown in the Figure 4.15. The user can zoom in the currently displayed image in order to see more details. Also, the user can zoom out in order to see the image compressed.

4.2.4 Quality Assurance

The XCTest is the default framework used for certain conditions during runtime. It provides all of the necessary features to make a complete testing session. We used XCTest for unit testing. We tested some of the ViewModels. We created a special class with multiple test cases for every tested ViewModel. The tests are, by default,

running on the target device of the project.

Also, we manually tested all of the application flows and features. The made steps are:

1. **Happy Flow:** We took the screens of the application one by one. First, we started with the *Home Screen*. We scrolled multiple times up and down. We opened all of the albums one by one. From a random album, we first opened the highlighted photo and after that one random photo and the oldest one. All photos were exported. We zoomed in and zoomed out. After that we deleted the photos. We renamed the album and returned to the *Home Screen*. We closed the application and opened it to verify the persistence of the changes. We repeated this steps, but first adding a photo from the phone galley and after that making one with the camera. The *Photo Viewer* was tested on both original and enhanced image. We moved to the *Create Album screen*. First we created an album adding multiple photos from the gallery and after that making multiple photos with the camera. We tested that screen in the same way as the one opened by pressing to an album. We moved to the Gallery Screen. We scrolled, opened first, last and random images, deleted them. After all the changes, we tested the persistence again. Another use case was to move from the *Home Screen* to the *Create Album* and back. We tested that the screen is still properly working. We repeated that step moving random between the screens.
2. **Bugs Hunting:** We tried to find some bugs that are less obvious. We tried to:
 - (a) scroll when we keep other finger on an album.
 - (b) press on two albums at the same time.
 - (c) press on an album when we are keeping other finger on random parts of the screen.
 - (d) press the buttons from the tab bar holding fingers random on the screen.
 - (e) to scroll vertically in home screen.
 - (f) press on two photos at the same time in Album Details.
 - (g) press *Upload* and *Open camera* buttons at the same time.
 - (h) scroll vertically on the Album Details sections.
 - (i) scroll horizontally on the Album Details sections that are not allowing it.
 - (j) delete a photo while we are zooming it.
 - (k) scroll vertically in the Photo Viewer Screen.
 - (l) export a photo while scrolling to change the photo.

- (m) zoomed in using four fingers.
- (n) zoomed out using three fingers.
- (o) press three images at the same time in gallery.
- (p) scrolled while pressing on two images at the same time.
- (q) tried to scroll down even if we were at the top of the page.
- (r) tried to scroll up even if we were at the bottom of the page.

Further development of the application can include the automation of the manual testing in order to save time.

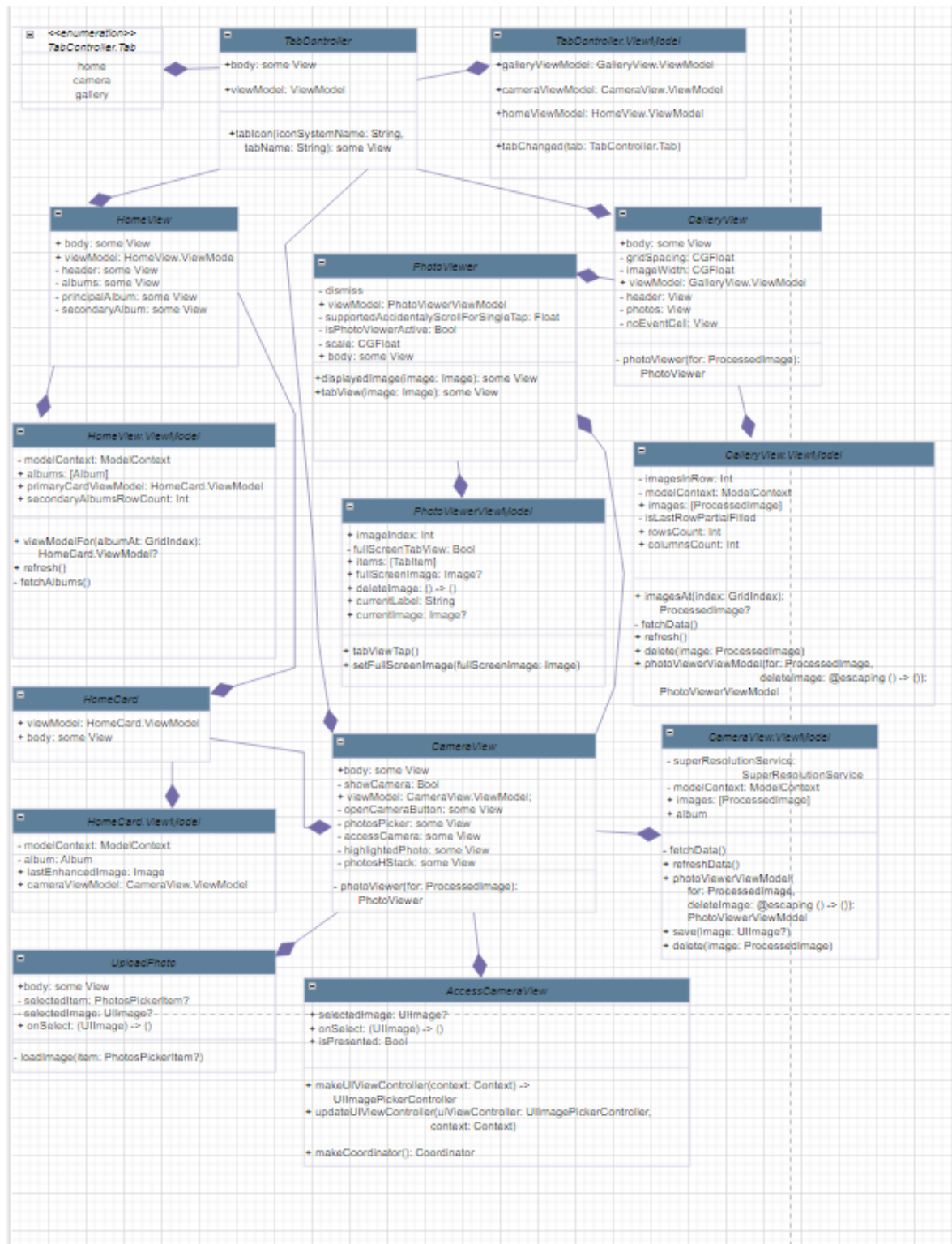


Figure 4.16: Presenter Class Diagram

Chapter 5

Conclusions and further possible improvements

The cameras and the monitors on which the pictures are displayed are in a continuous evolution, increasingly accelerated in recent years. In order for the old images not to lose their magic, they must be brought back to life. In this thesis, we discussed a new approach to enhance the old images increasing their resolution using an Artificial Neural Network. We detailed the implementation of two new models, starting from a state-of-the-Art one, changing the architecture of the model and obtaining faster models with comparative results. We presented a mobile application that offers to the users the whole experience of the better model.

Our research went through several stages, including the theoretical foundations of the deep learning. We reviewed classical optimizations functions possible improvements. A significant part was the searching and understanding of newer neural networks architectures.

We went through the different solutions of the Super Resolution problem and investigated the various approaches, including some classical algorithms, classical deep learning-based methods and GAN-based models. Starting from the innovations from the last category, we developed our models, named GhostSRGAN and SA GhostSRGAN. We used the benefits of the Ghost Bottlenecks and some Attention Mechanisms, creating models that are inferring realistic images in a short time. By integrating the first model, GhostSRGAN in our iOS mobile application, Ghost Enhancer, we proved that the model is suitable for environments with limited resources.

As in the case of any research project, there is always a huge potential of improvements. We started from the architecture of the SRGAN model and improved it, but there is a long list of improvements brought by the ESRGAN and EnhanceNet, proved to be very effective, that were not added in our network. Some small architectural changes may lead to better results. Also, we tried to improve our GhostSR-

GAN by using attention mechanisms. That approach failed to improve the accuracy of the model, but we think there is still much room for improvement, maybe changing the architecture a little bit more or using different attention mechanisms.

In conclusion, this work adds to the field of Computer Vision. We presented a solution for improving the quality of the images, having a huge potential in the field of photography and even a good solution for remastering images or videos.

Bibliography

- [Abr05] Ajith Abraham. Artificial neural networks. *Handbook of measuring system design*, 2005.
- [ADBB17] Kai Arulkumaran, Marc Peter Deisenroth, Miles Brundage, and Anil Anthony Bharath. Deep reinforcement learning: A brief survey. *IEEE Signal Processing Magazine*, 34(6):26–38, 2017.
- [ADIP21] Andrea Apicella, Francesco Donnarumma, Francesco Isgrò, and Roberto Prevete. A survey on modern trainable activation functions. *Neural Networks*, 138:14–32, 2021.
- [AML24] Hanadi Al-Mekhlafi and Shiguang Liu. Single image super-resolution: a comprehensive review and recent insight. *Frontiers of Computer Science*, 18(1):181702, 2024.
- [Cho17] François Chollet. Xception: Deep learning with depthwise separable convolutions. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 1251–1258, 2017.
- [DLHT15] Chao Dong, Chen Change Loy, Kaiming He, and Xiaoou Tang. Image super-resolution using deep convolutional networks. *IEEE transactions on pattern analysis and machine intelligence*, 38(2):295–307, 2015.
- [HA21] Saad Hikmat Haji and Adnan Mohsin Abdulazeez. Comparison of optimization techniques based on gradient descent algorithm: A review. *PalArch's Journal of Archaeology of Egypt/Egyptology*, 18(4):2715–2743, 2021.
- [JZZ⁺21] Yu Jin, Jiayi Zhang, Xiaodan Zhang, Huahua Xiao, Bo Ai, and Derrick Wing Kwan Ng. Channel estimation for semi-passive reconfigurable intelligent surfaces with enhanced deep residual networks. *IEEE transactions on vehicular technology*, 70(10):11083–11088, 2021.
- [KL18] Jun-Hyuk Kim and Jong-Seok Lee. Deep residual network with enhanced upscaling module for super-resolution. In *Proceedings of the IEEE*

- conference on computer vision and pattern recognition workshops*, pages 800–808, 2018.
- [LLWT15] Ziwei Liu, Ping Luo, Xiaogang Wang, and Xiaoou Tang. Deep learning face attributes in the wild. In *Proceedings of International Conference on Computer Vision (ICCV)*, December 2015.
- [LSK⁺17] Bee Lim, Sanghyun Son, Heewon Kim, Seungjun Nah, and Kyoung Mu Lee. Enhanced deep residual networks for single image super-resolution. In *Proceedings of the IEEE conference on computer vision and pattern recognition workshops*, pages 136–144, 2017.
- [LTH⁺17] Christian Ledig, Lucas Theis, Ferenc Huszár, Jose Caballero, Andrew Cunningham, Alejandro Acosta, Andrew Aitken, Alykhan Tejani, Johannes Totz, Zehan Wang, et al. Photo-realistic single image super-resolution using a generative adversarial network. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 4681–4690, 2017.
- [Mah20] Batta Mahesh. Machine learning algorithms-a review. *International Journal of Science and Research (IJSR)*.*[Internet]*, 9(1):381–386, 2020.
- [MBD⁺90] Tom Mitchell, Bruce Buchanan, Gerald DeJong, Thomas Dietterich, Paul Rosenbloom, and Alex Waibel. Machine learning. *Annual review of computer science*, 4(1):417–433, 1990.
- [ON15] Keiron O’shea and Ryan Nash. An introduction to convolutional neural networks. *arXiv preprint arXiv:1511.08458*, 2015.
- [REKL17] Haoyu Ren, Mostafa El-Khamy, and Jungwon Lee. Image super resolution based on fusing multiple convolution neural networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR) Workshops*, July 2017.
- [REKL18] Haoyu Ren, Mostafa El-Khamy, and Jungwon Lee. Ct-srcnn: cascade trained and trimmed deep convolutional neural networks for image super resolution. In *2018 IEEE Winter Conference on Applications of Computer Vision (WACV)*, pages 1423–1431. IEEE, 2018.
- [SAU19] Umme Sara, Morium Akter, and Mohammad Shorif Uddin. Image quality assessment through fsim, ssim, mse and psnr—a comparative study. *Journal of Computer and Communications*, 7(3):8–18, 2019.

- [SPY⁺22] Furkat Sultonov, Jun-Hyun Park, Sangseok Yun, Dong-Woo Lim, and Jae-Mo Kang. Mixer u-net: An improved automatic road extraction from uav imagery. *Applied Sciences*, 12:1953, 02 2022.
- [SSH17] Mehdi SM Sajjadi, Bernhard Scholkopf, and Michael Hirsch. Enhancenet: Single image super-resolution through automated texture synthesis. In *Proceedings of the IEEE international conference on computer vision*, pages 4491–4500, 2017.
- [SZ14] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556*, 2014.
- [TSL22] Yingjie Tian, Duo Su, Stanislao Lauria, and Xiaohui Liu. Recent advances on loss functions in deep learning for computer vision. *Neurocomputing*, 497:129–158, 2022.
- [VSP⁺23] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need, 2023.
- [WK22] Yang Wang and Hassan A Karimi. Perceptual loss function for generating high-resolution climate data. *Applied Computing and Intelligence*, 2(2):152–172, 2022.
- [WL⁺22] Zhong Wang, Tong Li, et al. A lightweight cnn model based on ghostnet. *Computational intelligence and neuroscience*, 2022, 2022.
- [WMZT20] Qi Wang, Yue Ma, Kun Zhao, and Yingjie Tian. A comprehensive survey of loss functions in machine learning. *Annals of Data Science*, pages 1–26, 2020.
- [WYDL18] Xintao Wang, Ke Yu, Chao Dong, and Chen Change Loy. Recovering realistic texture in image super-resolution by deep spatial feature transform. In *The IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2018.
- [WYW⁺18] Xintao Wang, Ke Yu, Shixiang Wu, Jinjin Gu, Yihao Liu, Chao Dong, Yu Qiao, and Chen Change Loy. Esrgan: Enhanced super-resolution generative adversarial networks. In *Proceedings of the European conference on computer vision (ECCV) workshops*, pages 0–0, 2018.

- [ZLL⁺18] Yulun Zhang, Kunpeng Li, Kai Li, Lichen Wang, Bineng Zhong, and Yun Fu. Image super-resolution using very deep residual channel attention networks. In *Proceedings of the European conference on computer vision (ECCV)*, pages 286–301, 2018.
- [ZZZ⁺19] Xining Zhu, Lin Zhang, Lijun Zhang, Xiao Liu, Ying Shen, and Shengjie Zhao. Generative adversarial network-based image super-resolution with a novel quality loss. In *2019 International Symposium on Intelligent Signal Processing and Communication Systems (ISPACS)*, pages 1–2. IEEE, 2019.